Friedrich-Alexander-Universität
Technische Fakultät

On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees

Seminar:

Cryptography in Secure Messaging: Understanding and Enhancing Signal

Agenda

01 Introduction

02 Tree-based group Diffie-Hellman

03 ART: Construction

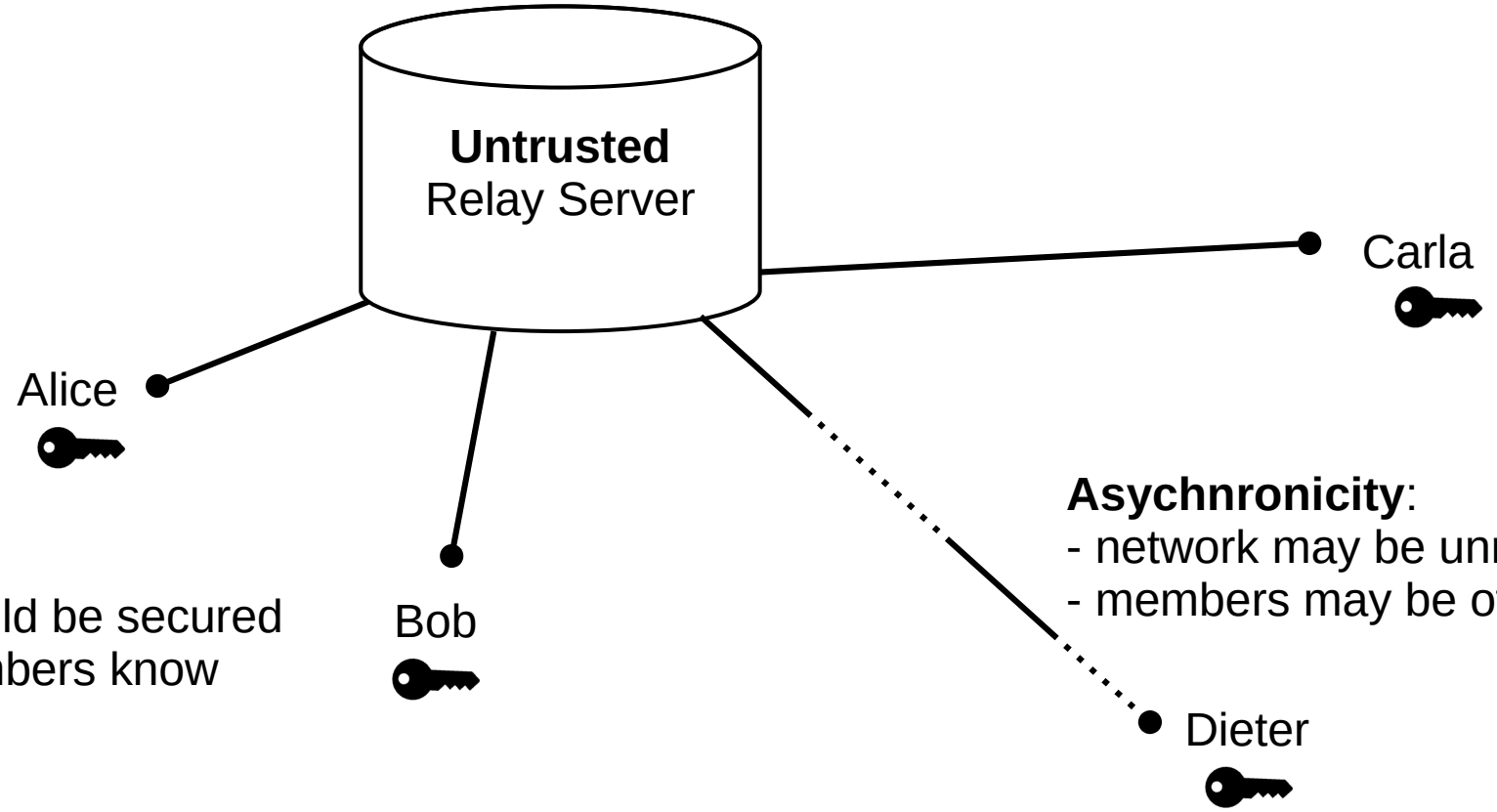
04 ART: Key Updates

05 Conclusion and Later Work

Introduction

Introduction

The usual setting for group communication:



Security:
- communication should be secured by secret(s) only members know

Asychnronicity:
- network may be unreliable
- members may be offline

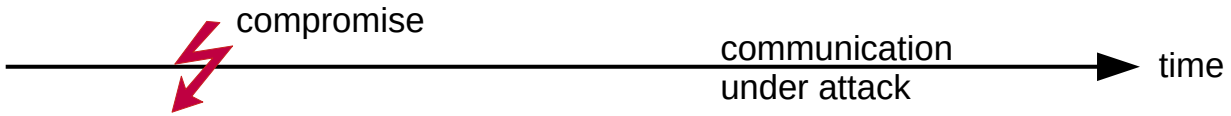
Introduction

Security Properties:

- **traditional**: confidentiality, integrity and availability (CIA)

- **Forward Security** (FS): „ *protects past communication* „

- **Post-Compromise Security** (PCS): „ *protects future communication* „



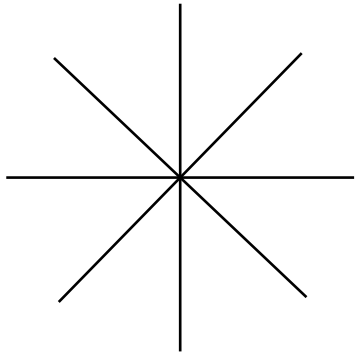
Introduction

Approaches providing security:

- Signal‘s **Sender Keys** variant

  + efficient

  - provides neither FS nor PCS

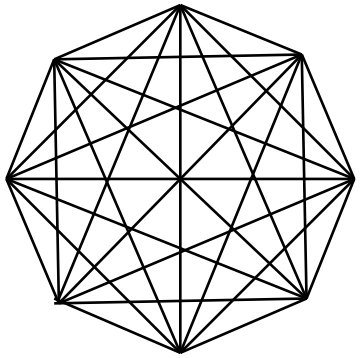


group topology for 8 members

- **pairwise ratcheting** between all members

  + provides FS and PCS

  - rather inefficient regarding large group size n
    → O(n) for sending



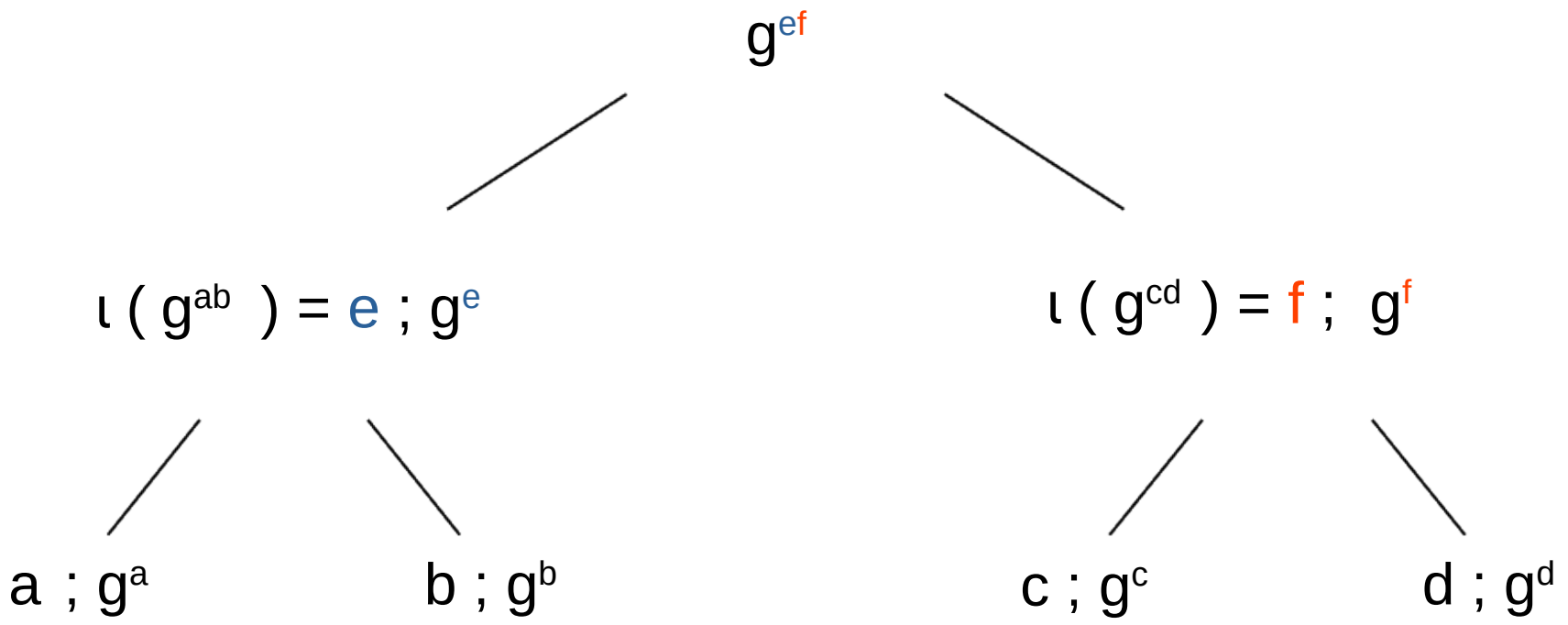
Tree-based group Diffie-Hellman

Tree-based group Diffie-Hellman

binary tree, 4 members

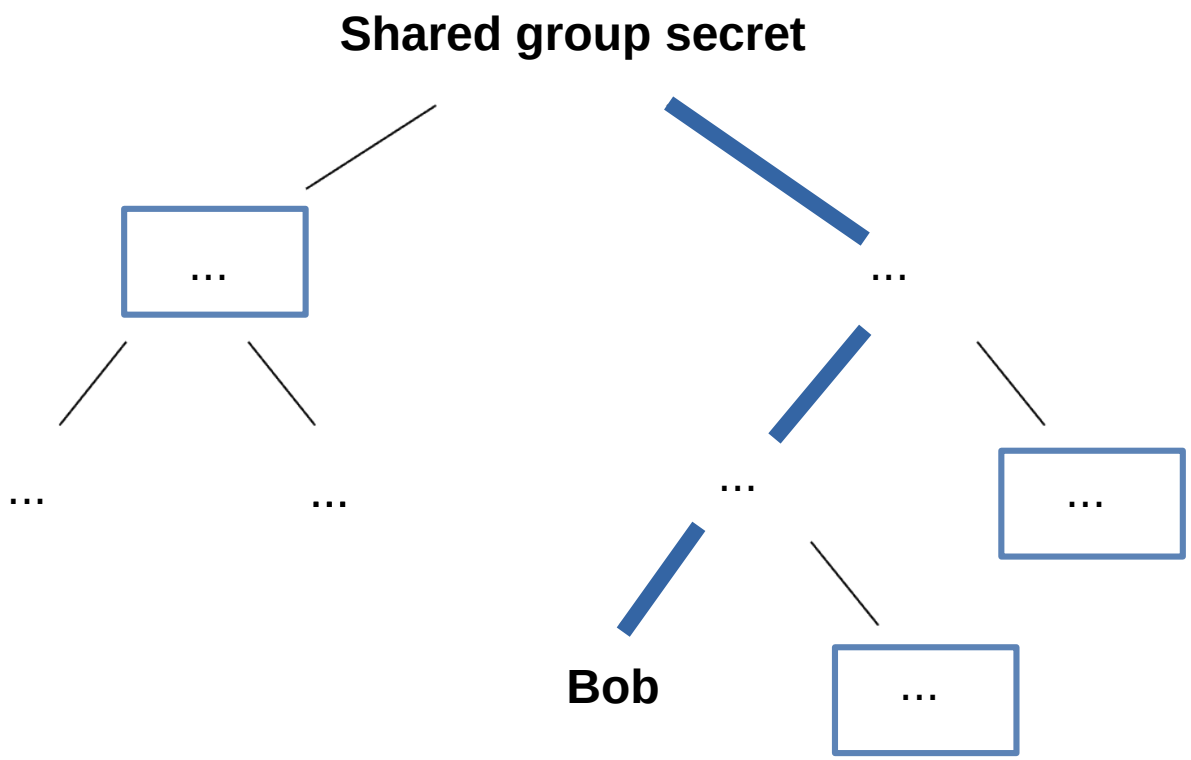
g is the generator used in the DH-group

ι (.) is a mapping from integers to group elements



Tree-based group Diffie-Hellman

- Bob wants to compute the shared group secret
- He must compute all the node keys **along his path** to the root
- he has to know all the **public keys** at the sibling nodes of his path (**co-path**)

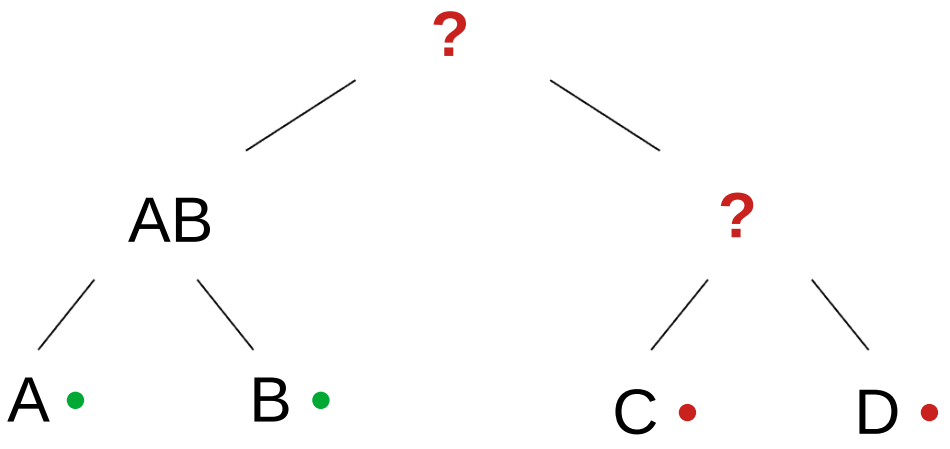


Tree-based group Diffie-Hellman

Diffie-Hellman Trees have nice properties:

- key at the root of the tree only computable by knowing **one of the leaves‘ private keys**
- **logarithmic depth,** O(log n) with n as group size

- **Problem**: DH key exchange is synchronous



Tree-based group Diffie-Hellman

Demands for group messaging design:

- **Asynchronicity**: derive leaf key after group initialization
- **security**: nobody else can derive that key

The idea:

- use of **prekeys** (epheremal public keys) stored in advance at a server
- group initiator can fetch these keys to perform an **intial key exchange** while members may be offline

→ **Aynchronous Ratcheting Tree** (ART)

ART - Construction

ART - Construction

Alice (A) wants to **create a group** with 3 other members (B, C and D) ...

1. creates fresh **setup key** pair

   (suk, SUK = g^{suk})

2. fetches **prekeys** EK_u and **identity keys** IK_u for all members u

- perform an **initial key exchange** with any strong one-round **Authenticated Key Exchange** (AKE)

  initial leaf key for member u: $\lambda_u = \mathit{KeyExchange}(ik_A, IK_u, suk, Ek_u)$

  $= \mathit{KeyExchange}(ik_u, IK_A, ek_u, SUK)$

ART - Construction

- A can now construct the intial tree …
- Afterwards she can broadcast:

  1. the public **prekeys** EK_u and public **identity keys** IK_u
  2. the **public setup key** SUK
  3. the **tree** T of public keys
  4. a **signature** of (1., 2., 3.) under her private identity key ik_A

ART - Construction

Any member **u** going online retrieves this message and …

… **verifies the signature** using IK_A

… computes its **initial leaf key:** $\lambda_u = \mathit{KeyExchange}(ik_u, IK_A, ek_u, SUK)$

… uses T to compute the shared group secret (**tree key tk**)

ART - Construction

Initial Tree

$$g^{\iota(g^{\lambda_1^A \lambda_1^B})\iota(g^{\lambda_1^C \lambda_1^D})}$$

$$\iota(g^{\lambda_1^A \lambda_1^B}) \; ; \; g^{\iota(g^{\lambda_1^A \lambda_1^B})} \qquad \iota(g^{\lambda_1^C \lambda_1^D}) \; ; \; g^{\iota(g^{\lambda_1^C \lambda_1^D})}$$

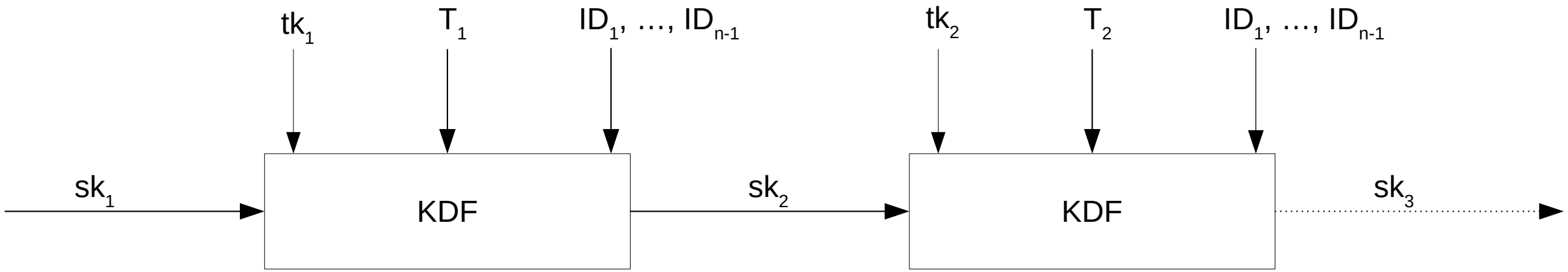
$$\lambda_1^A \; ; \; g^{\lambda_1^A} \qquad \lambda_1^B \; ; \; g^{\lambda_1^B} \qquad \lambda_1^C \; ; \; g^{\lambda_1^C} \qquad \lambda_1^D \; ; \; g^{\lambda_1^D}$$

https://eprint.iacr.org/2017/666.pdf

ART – Key Updates

ART – Key Updates

- the phases between a key update will be called **stages**
- each stage holds a **stage key sk**

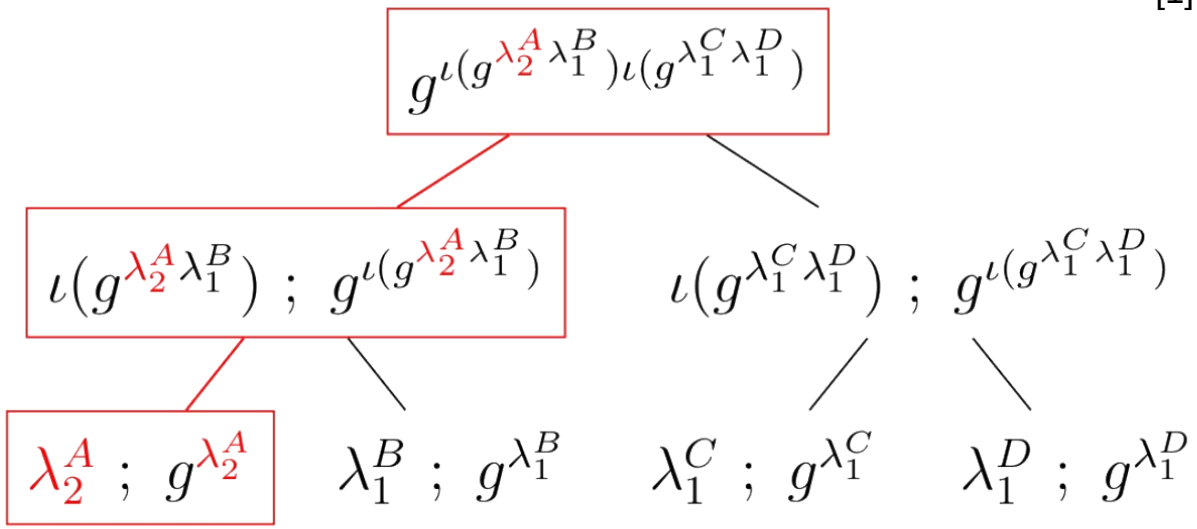


→ **stage key chaining**: $sk_1, \ldots, sk_t, \ldots$ build a hash chain

ART – Key Updates

- Any member can **update** its private leaf key at any time

[1]



A has updated its private leaf key and all the node keys along its path to the root

- A then broadcasts her **update message**:

  - new **public keys m**

  - $MAC_k(m)$ $k = f(sk)$

m || MAC →

[1] https://eprint.iacr.org/2017/666.pdf

ART – Key Updates

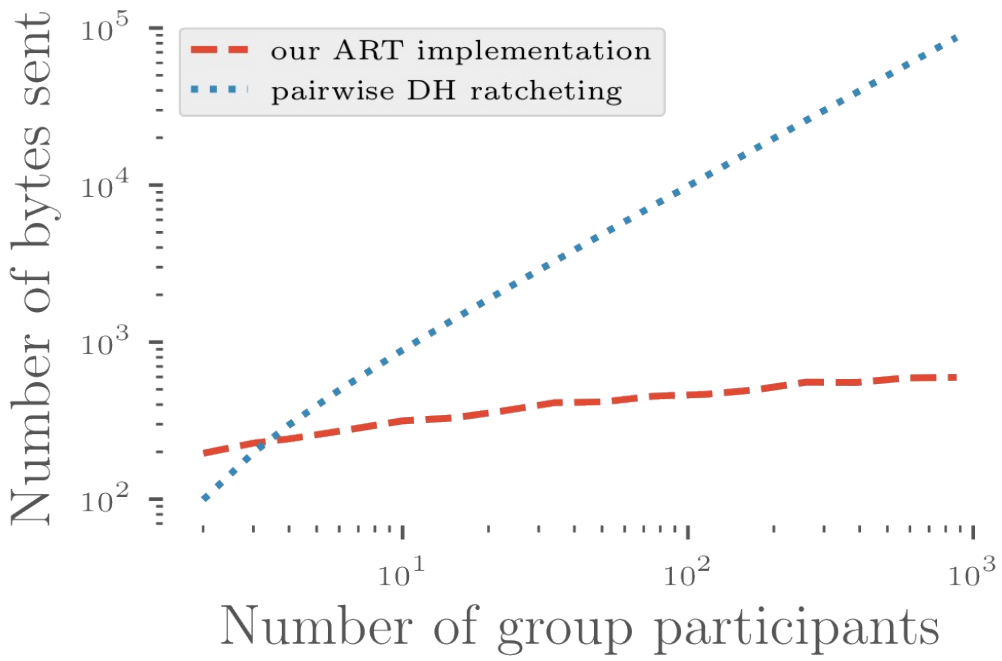
m || MAC →

On reception:

- **MAC verify**
- update tree to compute new **tree key** → new **stage key sk**

Key Updates - Properties:

- **Efficient**
- **Asynchronous**
- **Secure**



https://eprint.iacr.org/2017/666.pdf

Conclusion and Later Work

Conclusion

Asynchronous Ratcheting Tree (ART)

- **asynchronous**
- **efficient**
- **strong security properties**

→ Message Layer Security (MLS) protocol

Thank you for your attention :)

Source:
On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees
Cryptology ePrint Archive, Paper, 2017
Katriel Cohn-Gordon and Cas Cremers and Luke Garratt and Jon Millican and Kevin Milner
https://eprint.iacr.org/2017/666